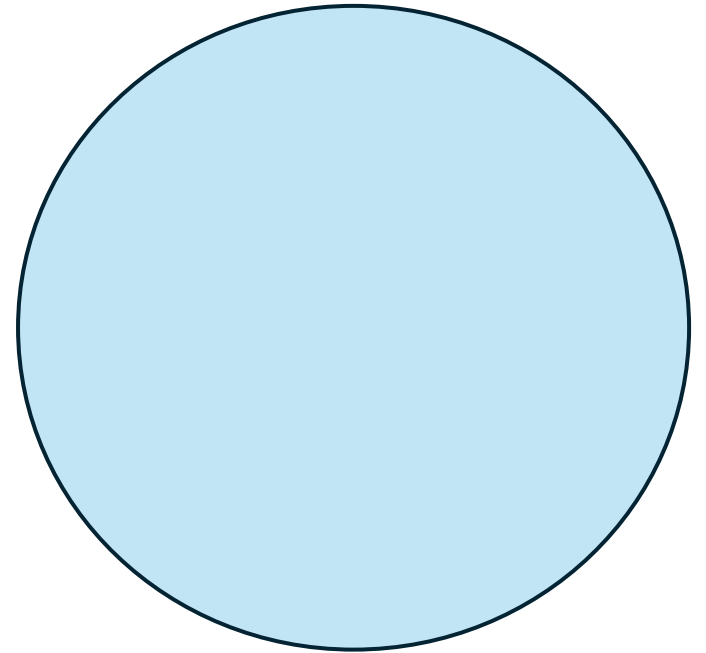


(Hyper)-Graph Problems via Global Queries

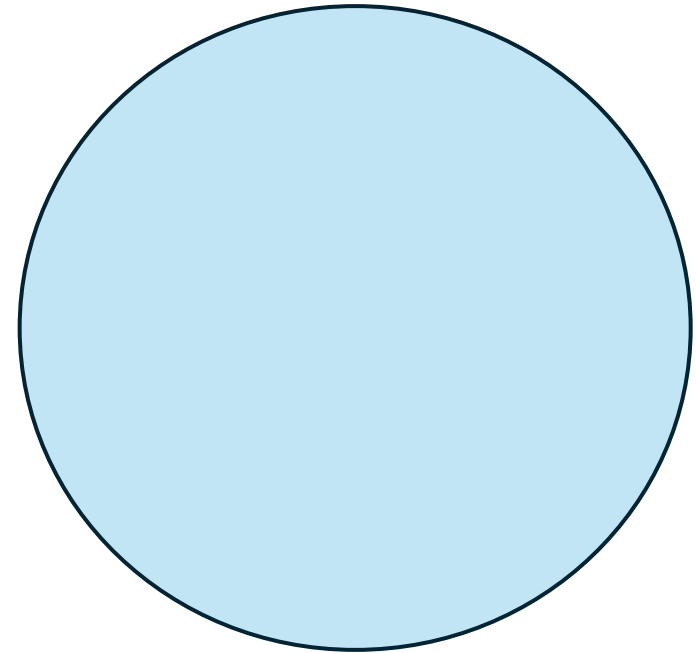
Query Access Model (QAM) for algorithms

- Access object via certain queries
- Solve algorithmic prob, usually appx
- Usually focus on query complexity
- Upper bounds / Lower bounds



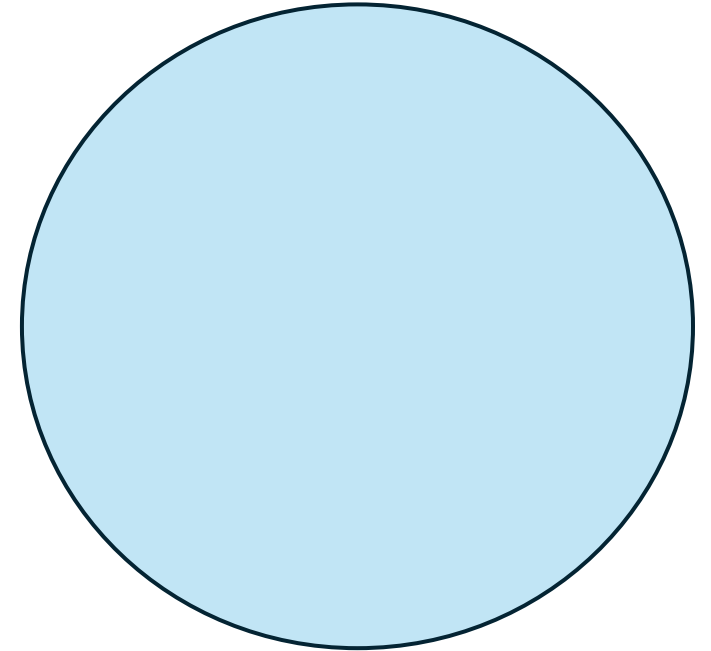
“Local” Queries: Sublinear Algorithms

- Random Access (vertex / edge)
- Get “local” inf about object
 - degree
 - random neighbor



“Global” Queries

- Vertex Set known
 - **Query** info about **subset** of vertices
 - Is the set an independent set?
 - How many edges are in the cut set?
 - What's the shortest path length between s & t?



Why?

Considerations

- Query Complexity
- Adaptivity / Round complexity
- Deterministic / Randomized
- Efficiency with respect to time

Rest of my time

- An **incomplete** survey
- Classics [40s to 60s]
- Graph Learning [90s to 10s]
- Graph properties [recent-er]

Classics [40s to 60s]

Reconstructing vectors

- Unknown object: n-dimensional vector $x \in \{0,1\}^n$
- Two Query Models: both take input $S \subseteq [n]$
 - (1) Hit(S) = YES if $x_i = 1$ for some $i \in S$, NO otherwise
Group Testing Dorfman '43
 - (2) Sum(S) = $\sum_{i \in S} x_i$ Coin Weighing Shapiro '60

Goal: reconstruct x

Group Testing

- If nothing else is known, n -queries needed

every query gives only one bit of info

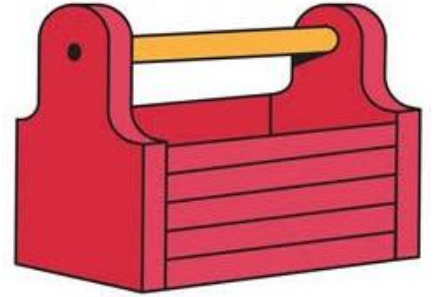
- Usually focus on at most d ones case

need $\log \binom{n}{d} \approx d \log n$ queries

Group Testing ($d=1$)

Tool: Binary Search

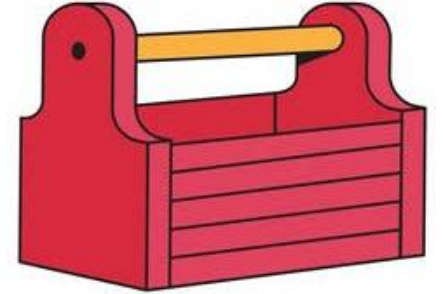
- Take arbitrary half A of $[n]$ and query $\text{Hit}(A)$
- if YES, recurse on A ; o/w recurse on $[n] \setminus A$
- Optimal in Query Complexity
- Adaptive



Group Testing (d=1)

Tool: “non-adaptive” binary search

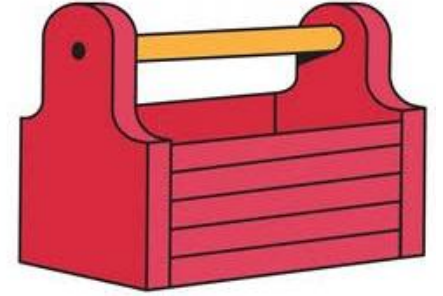
0	0	0	1	1	1	1
0	1	1	0	0	1	1
1	0	1	0	1	0	1



Group Testing (d=1)

Tool: “non-adaptive” binary search

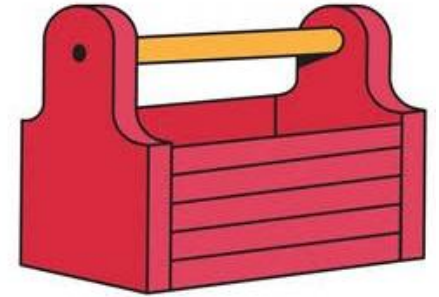
Y	0	0	0	1	1	1	1
N	0	1	1	0	0	1	1
Y	1	0	1	0	1	0	1



Group Testing (general d)

- If adaptivity not an issue, $O(d \log n)$ repeated binary search
- Non-adaptive?

Group Testing (general d)

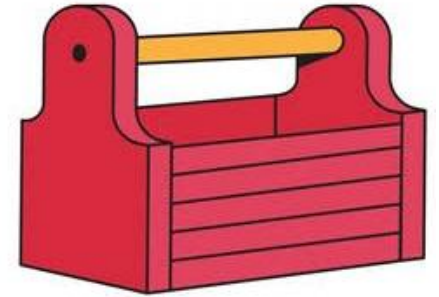


Tool: random sampling

- Sample K subsets picking each i with probability $1/d$
 - Zero all coordinates which participate in a NO query
 - Rest all 1
-
- If $x_i = 1$, we report 1 on i
 - If $x_i = 0$ $\Pr[\text{a sample } R \text{ zeros } i] = \Pr[i \in R] * \Pr[\text{no } 1 \text{ in } R]$

$$\frac{1}{d} \quad \left(1 - \frac{1}{d}\right)^d \geq \frac{1}{4}$$

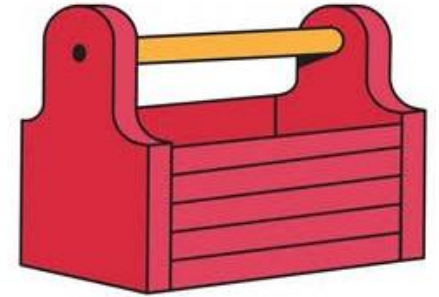
Group Testing (general d)



Tool: random sampling

- Sample K subsets picking each i with probability $1/d$
 - Zero all coordinates which participate in a NO query
 - Rest all 1
-
- If $x_i = 1$, we report 1 on i
 - If $x_i = 0$ $\Pr[\text{a sample } R \text{ zeros } i] \geq 1/4d$

Group Testing (general d)



Tool: random sampling

- Sample K subsets picking each i with probability $1/d$
 - Zero all coordinates which participate in a NO query
 - Rest all 1
-
- If $x_i = 1$, we report 1 on i
 - If $x_i = 0$ $\Pr[\text{we don't report 0}] \leq \left(1 - \frac{1}{4d}\right)^K$
- If $K = 4d \ln \left(\frac{n}{\delta}\right)$, then algo correct with prob $1 - \delta$

Group Testing (general d)

- There is an $O\left(d \log\left(\frac{n}{\delta}\right)\right)$ query randomized algo which on any x succeeds with prob $1 - \delta$
- If $\delta < \frac{1}{\binom{n}{d}} \sim \frac{1}{n^d}$, then by union bound there *exists*
 $O\left(d \log\left(\frac{n}{\delta}\right)\right) \sim O(d^2 \log n)$ queries whose answers determine x

These can be made explicit using connections to error correcting codes

Coin Weighing

- Unknown Boolean vector $\mathbf{x} \in \{0,1\}^n$
- SUM Query(S) $\mapsto \sum_{i \in S} x_i$
- *Question:* Learn $\mathbf{x}$

- Information theoretically: $\geq n/\log_2 n$
- Non-adaptively $\geq \frac{2n - o(n)}{\log_2 n}$

$$\mathbf{A} \in \{0, 1\}^{\ell \times n}$$

$$\ell = \frac{2n + o(n)}{\log_2 n} \text{ suffices!}$$

$\mathbf{x}$

=

$\mathbf{Ax}$

$\mathbf{Ax}$ determines $\mathbf{x}$

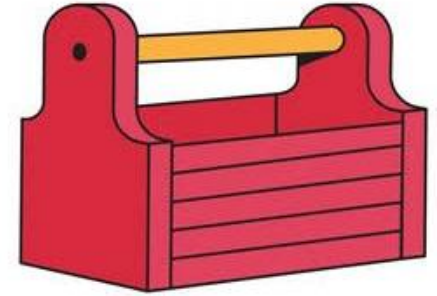
Soderbergh, Shapiro (1960)

Erdos, Renyi (1963)

Cantor, Mills (1966)

Lindstrom (1965)

Coin Weighing



- Unknown Boolean vector $\mathbf{x} \in \{0,1\}^n$
- SUM Query(S) $\mapsto \sum_{i \in S} x_i$
- *Question:* Learn $\mathbf{x}$
 - There are efficient $O(n/\log n)$ query non-adaptive algorithms Cantor-Mills, Lindstrom (66)
 - If $\mathbf{x}$ has d ones, there exists $O(\frac{d \log(\frac{n}{d})}{\log d})$ query non-adaptive algorithms
 - $O(\frac{d \log(\frac{n}{d})}{\log d})$ query *efficient* adaptive algorithms Bshouty (2009),
Bshouty-Mazzawi (2011)
 - If $\mathbf{x}$ is weighted, non-negative with $\text{supp} = d$,
there is $O(\frac{d \log(\frac{n}{d})}{\log d})$ query randomized adaptive algo Choi (2013)

Coin Weighing (Ideas?)

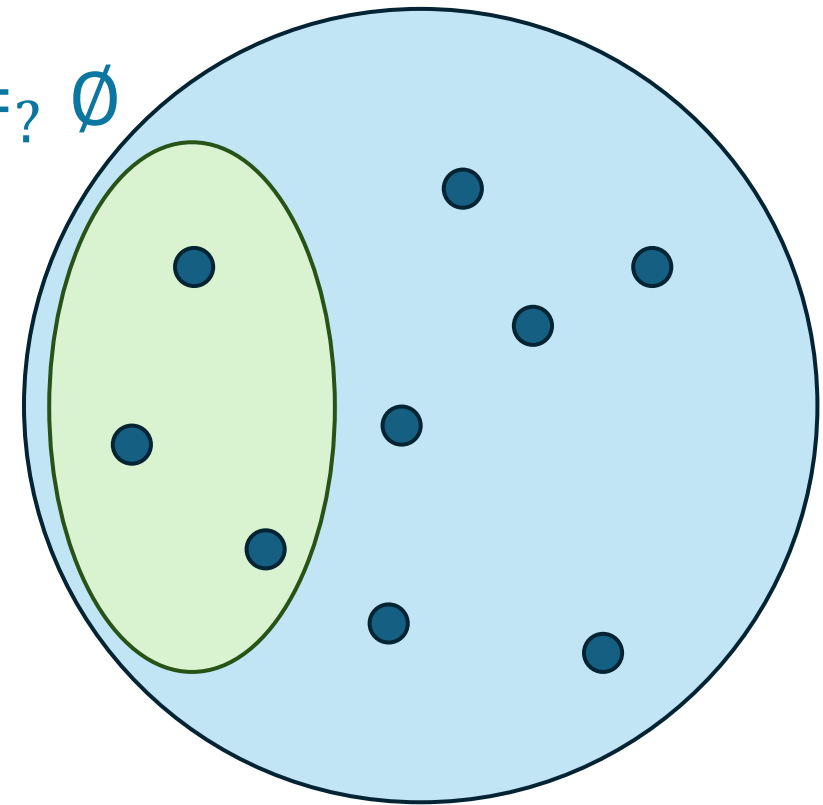
Graph Learning [90s to 10s]

Undirected Graphs

$$E[S] := \{uv \in E : u \in S, v \in S\}$$

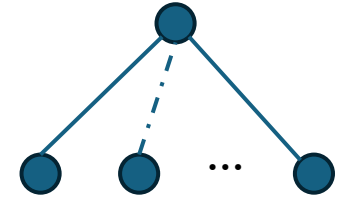
$$\partial S := \{uv \in E : u \in S, v \notin S\}$$

- Unknown object: undirected graph with known V
- Edge Detecting / Ind Set $S \mapsto E[S] =? \emptyset$
- Additive $S \mapsto |E[S]|$
- CUT $S \mapsto |\partial S|$
Submodular function
- Goal: reconstruct the edges of the graph



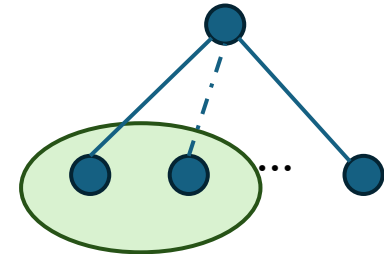
Observations

- Independent Set Queries generalizes group testing
- Additive Queries generalizes coin weighing
- CUT queries $\equiv$ Additive queries after degrees
- Graph can be learnt in $O(n^2 / \log n)$ ADD/CUT queries...
- ...this is optimal



$$\sum_{v \in S} \deg(v) = 2|E[S]| + |\partial S|$$

Learn every vertex's nbrhood*

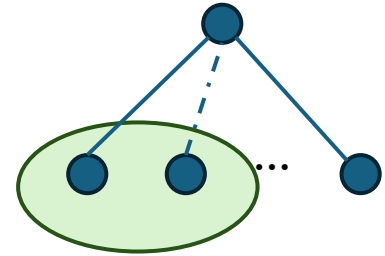


Focus: case when graph has only m edges

$$\Omega(m \log n) \text{ IS}$$
$$\Omega(m \log n / \log m) \text{ CUT/ADD}$$

Independent Set Queries with m edges

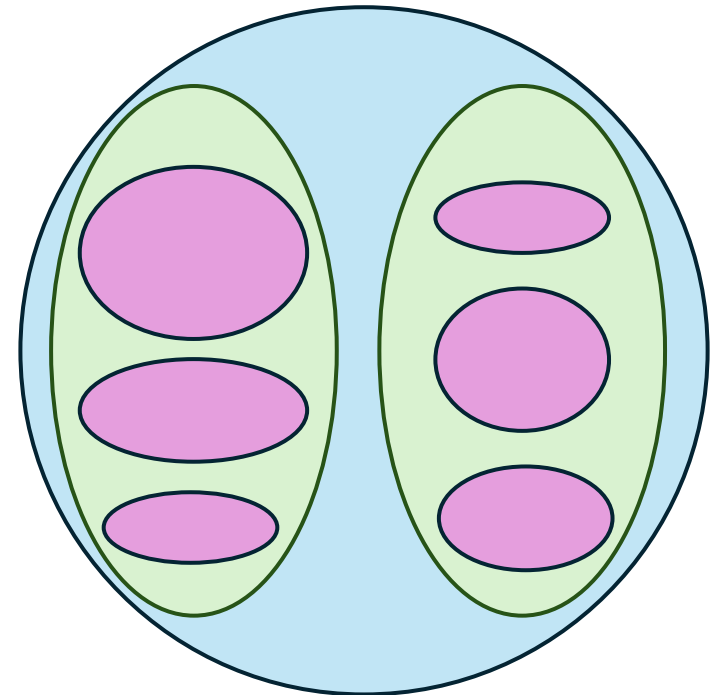
- $\Omega(m \log n)$ is the best one can hope for
- Can't use “learn vrtx nbrhood” trick since “green zone” may not be independent



Angluin-Chen (2004)

Exercise

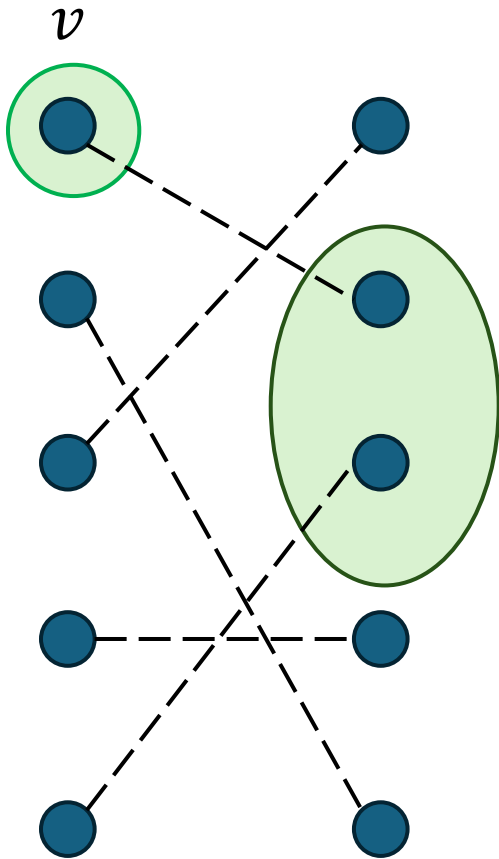
- Recursively learn two halves
- Each half can be $O(\sqrt{m})$ -colored
- Learn edges between Ind Sets using group testing costing $O(\log n)$ per edge



ADD/CUT queries with m edges

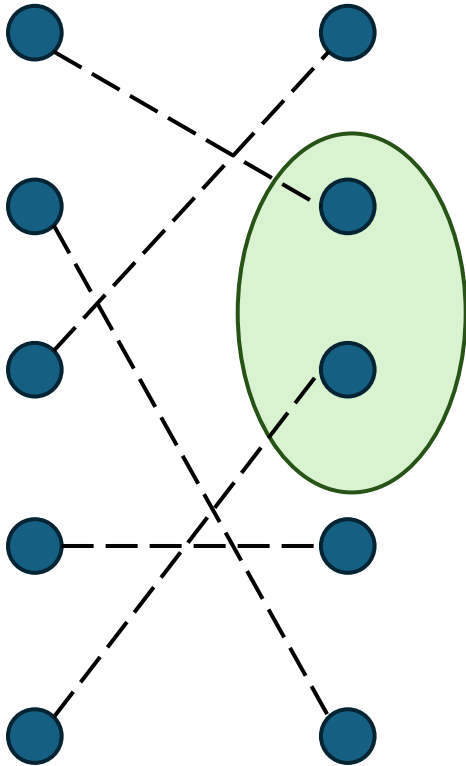
- With ADD/CUT queries, lower bound: $\Omega(m \log n / \log m)$
- In particular, for sparse graphs: $\Omega(n)$
- This is also the best known algorithmically!
 - $O\left(\frac{m \log n}{\log m}\right)$ query algorithm for m-edge graphs
 - Choi, Kim (2008)
 - Bshouty, Mazzawi (2009)
 - Mazzawi (2010)
 - Bshouty, Mazzawi (2011)
 - Choi (2013)
 - Even works with weights

Learning a Hidden Matching



- v needs to learn a Boolean vector with exactly one 1
- There are $\sim \log_2 n$ subsets $S_1, \dots, S_{\log n}$ such that knowing $\text{ADD}(v, S_i)$'s will reveal edge inc. on v
- Key: *these subsets don't depend on v*
- What we need: for each S_i , which vertices have edge to S_i

Learning a Hidden Matching



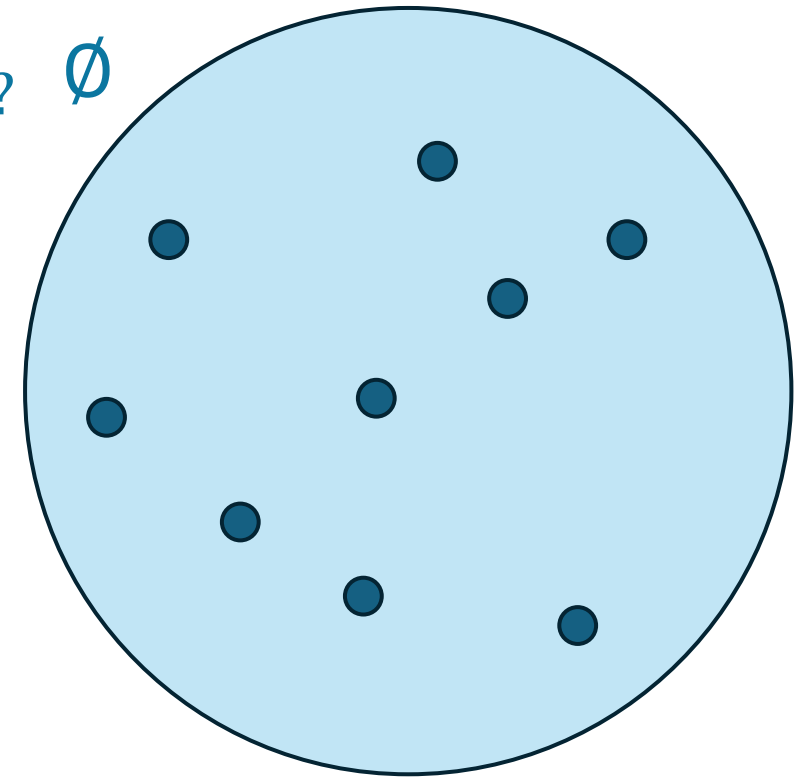
- What we need: for each S_i , which vertices have edge to S_i
- $\mathbf{x}^{(i)} \in \{0,1\}^n$ where $x_j^{(i)} = 1$ iff j has an edge to S_i
- Sum query on this unknown $\mathbf{x}$ with ADD query
- Can learn $\mathbf{x}^{(i)}$ in $\sim \frac{2n}{\log_2 n}$ queries
- Doing this for all $\sim \log_2 n$ sets gives the result

Hypergraphs

$$E[S] := \{e \in E : e \subseteq S\}$$

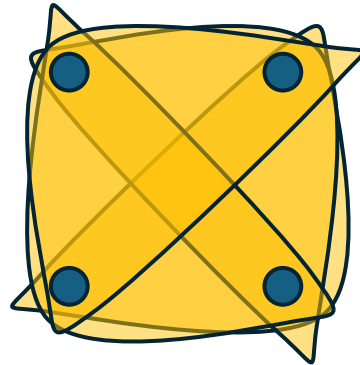
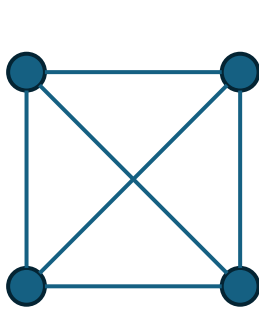
$$\partial S := \{e \in E : e \cap S \neq \emptyset, e \cap \bar{S} \neq \emptyset\}$$

- Unknown object: undirected **hypergraph** with known V
 - Edge Detecting / Ind Set $S \mapsto E[S] =? \emptyset$
 - Additive $S \mapsto |E[S]|$
 - CUT $S \mapsto |\partial S|$
Submodular function
- Goal: reconstruct the hyperedges



Observations

- Additive and CUT queries are **not** equivalent
- In fact, CUT queries **cannot** reconstruct hypergraphs!



Exercise?

- On the other hand, can learn m -hyperedge hypergraph in $O(mn \log n)$ ADD queries

Angluin-Chen (2006)

Graph Properties [2018 - ...]

Two papers from ITCS 2018

Unknown object: undirected graph with known V

Beame, Har-Peled, Ramamoorthy, Rashtchian, Sinha

Edge Detecting / Ind Set $S \mapsto E[S] =? \emptyset$

Goal: Estimate number of edges

Chen, Waingarten, Levi (2020)

Bhattacharya-Bishnu-Ghosh-Mishra (2020)

Assadi-Chakrabarty-Khanna (2022)

Kupfer-Nisan (2022)

Addanki-McGregor-Musco (2022)

Dell-Lapinskas-Meeks (2024)

Adar-Hotam-Levi (2026)...

Rubinstein, Schramm, Weinberg

CUT $\equiv$ ADD queries $S \mapsto |\partial S|$

Goal: Return minimum cut

Motivation:

special case of submodular
function minimization (SFM)

SFM can be solved in $\tilde{O}(n^2)$ queries

Very brief lay of the land with CUT queries

	Unweighted Undirected Graphs	Weighted Undirected Graphs	Unweighted Directed Graphs	Unweighted Hypergraphs
Connected? (mincut = 0 or not)	$O(n)$ Las Vegas [Apers et al 2022]	$O(n)$ Las Vegas [Liao, C. 2024]	$O(n^2 / \log n)$ [folklore]	$O(n \log n)$ [exercise] $O(n)$ [Liao, C 2026]
(Global) Min-Cut	$O(n)$ [Apers et al 2022]	$O(n \text{ polylog } n)$ [Mukhopadhyay-Nanongkai 2020]	$O(n^2 / \log n)$ [folklore]	$\tilde{O}(n^2)$ [symmetric SFM]
(s,t) Min-Cut	$\tilde{O}(n^{3/2})$ [Kenneth-Mordoch, Krauthgamer, 2026]	$O(n^2)$ [learn the whole graph]	$O(n^2 / \log n)$ [folklore]	$\tilde{O}(n^2)$ [SFM]

Very brief lay of the land with CUT queries

	Unweighted Undirected Graphs	Weighted Undirected Graphs	Unweighted Directed Graphs	Unweighted Hypergraphs
Connected? (mincut = 0 or not)	$O(n)$ Las Vegas [Apers et al 2022]	$O(n)$ Las Vegas [Liao, C. 2024]	$O(n^2 / \log n)$ [folklore]	$O(n \log n)$ [exercise] $O(n)$ [Liao, C 2025]
(Global) Min-Cut	$O(n)$ [Apers et al 2022]	$O(n \text{ polylog } n)$ [Mukhopadhyay-Nanongkai 2020]	$O(n^2 / \log n)$ [folklore]	$\tilde{O}(n^2)$ [symmetric SFM]
(s,t) Min-Cut	$\tilde{O}(n^{3/2})$ [Kenneth-Mordoch, Krauthgamer, 2026]	$O(n^2)$ [learn the whole graph]	$O(n^2 / \log n)$ [folklore]	$\tilde{O}(n^2)$ [SFM]

over to Louie...